

Module 9 Week B — Stretch: KG Critic — A 4-Stage Cypher Verifier Cascade

Honors Track. This stretch is for learners who have completed all core assignments, are On Track or Advanced, and are attending consistently. Stretch counts toward Honors distinction; it is not required for program completion.

Due: Thursday end of day. Resubmissions are accepted through Saturday of the assignment week.

Prerequisite: Complete the Module 9 Week B Applied Lab through `disambiguate()` — the Critic reuses the Lab’s Identity Discipline (the `:Entity {id}` uniqueness constraint) and the same recipe-graph schema. The M8 stretch (query router) is optional context for the writeup section on multi-agent composition; it is not required to complete this stretch.

What you’re building

A **KG Critic** — a verifier that takes a structured claim about the recipe graph (a triple `(subject_id, predicate, object_id)`) and adjudicates it against the graph via a four-stage cascade. The signature you implement in `critic/verify.py`:

```
def verify_claim(driver, claim: tuple[str, str, str]) -> Verdict:
    ...
```

`Verdict` is a frozen dataclass shipped in `critic/verdict.py`:

- `verdict` — one of `"supported"`, `"entailed"`, `"contradicted"`, `"unsupported"`
- `evidence_paths` — list of node-id chains that justify the verdict
- `confidence` — float pinned by the cascade stage that fired

A KG critic is a first-class verification primitive in production retrieval systems. When an upstream component emits a structured claim, the critic adjudicates it against the graph rather than trusting the generated text. The cascade operationalizes one of the load-bearing answers to “what is a KG for?” — verification under the open-world assumption, with explicit, principled abstention.

The cascade runs the four stages **in order**. The first stage that fires returns; later stages do not run. Stage ordering matters — see Common Pitfalls.

Setup

Windows users. Commands below assume Bash/Zsh (macOS/Linux/WSL2/Git Bash). On PowerShell or CMD, activate your venv with `.venv\Scripts\Activate.ps1` (PowerShell) or `.venv\Scripts\activate.bat` (CMD), and confirm Docker Desktop is running with the WSL2 backend before `docker compose up -d`. Full cross-platform command tables are in the Pre-Work install plan.

- Fork github.com/LevelUp-Applied-AI/m9-s9b to your personal GitHub account.
- Enable Actions on your fork (Actions tab → “I understand my workflows, go ahead and enable them”). One-time per fork.
- Clone your fork and create the working branch:

```
git clone https://github.com/<your-username>/m9-s9b.git
cd m9-s9b
git checkout -b stretch-9b-tue-kg-critic
docker compose up -d
pip install -r requirements.txt
python load_fixture.py
pytest tests/ -v
```

`load_fixture.py` loads the frozen ~31-node verifier-eval subgraph and asserts the acceptance counts. The unmodified starter is expected to fail until you wire up the cascade in `critic/verify.py`.

Stage 1 — Direct EXISTS

The first stage asks the cheapest possible question: *is this claim already in the graph?*

- For a **relationship predicate** (`USES_INGREDIENT`, `OF_CUISINE`, `BY_AUTHOR`, `REQUIRES_TECHNIQUE`), this is a MATCH on the directed edge between the two `:Entity` nodes with the matching ids:

```
MATCH (h:Entity {id: $subject_id})-[:USES_INGREDIENT]->(t:Entity {id: $object_id})
RETURN h.id AS h, t.id AS t LIMIT 1
```

- For the **type predicate**, this is the reflexive case — `subject_id == object_id`, equivalently a depth-0 `[[:SUBCLASS_OF*0..0]]` traversal.

If the row exists, return:

```
Verdict("supported", [(subject_id, object_id)], 1.0)
```

The relationship-type filter cannot be a Cypher parameter in vanilla Cypher. Template-substitute the predicate string **only** after validating it against the known relationship-type set (`SCHEMA_CONSTRAINTS`). The subject and object ids must always flow through `$param` slots — never f-string interpolation.

Stage 2 — Hierarchical entailment via `[[:SUBCLASS_OF*1..]]`

The hierarchical entailment stage is the reason this assignment exists. Without it, recall on the `entailed-only` subset collapses to zero — a critic that only does a direct EXISTS check has no way to recognize that a claim is *implied* by the graph even when it isn’t directly asserted.

Worked example — a `type` claim. The graph asserts:

- `([:Cuisine {id: 'cuisine:sichuan'}]-[:SUBCLASS_OF]->[:Cuisine {id: 'cuisine:chinese'}]-[:SUBCLASS_OF]->[:Cuisine {id: 'cuisine:asian'}])`

The graph does **not** directly assert `(cuisine:sichuan, type, cuisine:asian)`. But the chain entails it: Sichuan is Chinese is Asian. Stage 2 is scoped to the `type` predicate only — non-`type` predicates (e.g., `OF_CUISINE`, `USES_INGREDIENT`) are not entailed through `:SUBCLASS_OF` and fall through to Stage 4.

The entailment traversal for the `type` claim above is:

```
MATCH path = (h:Entity {id: $subject_id})-[:SUBCLASS_OF*1..]->(t:Entity {id: $object_id})
RETURN [n IN nodes(path) | n.id] AS chain LIMIT 1
```

The `*1..` floor (depth ≥ 1) keeps this stage distinct from Stage 1’s depth-0 reflexive case. Build the `evidence_paths` from the actual node ids visited along the chain and return:

```
Verdict("entailed", [tuple_of_ids_along_chain], 0.7)
```

The `[[:SUBCLASS_OF*1..]]` variable-length pattern is the load-bearing operator for Stage 2. Its direction matters — see Common Pitfalls.

Stage 3 — Domain/range contradiction

The contradiction stage catches claims that violate the schema’s structural constraints — the predicate exists, but it is being applied to the wrong kind of entity on one side or the other.

Worked example. The claim `(ingredient:ginger, OF_CUISINE, cuisine:asian)` looks well-formed, but the schema says `:OF_CUISINE` is `([:Recipe]-[:OF_CUISINE]->[:Cuisine])`. The subject `ginger` carries the `:Ingredient` label, not `:Recipe` — the claim violates the predicate’s domain.

The starter ships a `SCHEMA_CONSTRAINTS` dict mapping each relationship predicate to its `(expected_source_label, expected_target_label)` pair, plus a `_labels_of(driver, node_id)` helper that returns a node’s non-`:Entity` labels via parameterized Cypher. Read the predicate’s expected labels, fetch the actual labels of the subject and object, and if either side disagrees, return:

```
Verdict("contradicted", [(subject_id, object_id)], 0.8)
```

The `type` predicate is intentionally absent from `SCHEMA_CONSTRAINTS` — type membership is governed by the `SUBCLASS_OF` hierarchy (Stage 2), not by a fixed label pair. Skip Stage 3 for `type` claims and proceed to Stage 4.

Stage 4 — Abstain (the honest “no”)

If none of Stages 1–3 fire, the cascade abstains:

```
Verdict("unsupported", [], 0.5)
```

This is the open-world stance. Absence of evidence is not evidence of absence — the graph may simply not know. `unsupported` is a principled answer, not a guess. Over-flagging a claim as `contradicted` when it merely lacks evidence is a worse error than abstaining, because `contradicted` carries a higher confidence and a stronger commitment.

Tests

Run the autograder:

```
pytest tests/ -v
```

The autograder loads the 40-claim eval set (20 `supported`, 10 `entailed-only`, 10 `contradicted`), runs `verify_claim` over each, and asserts per-class precision and recall thresholds pinned against the reference 4-stage cascade — see the test docstrings for the exact values. Local pytest also exercises a parameterized-Cypher AST check that rejects f-string interpolation of any claim component into a Cypher string.

Record your per-class precision and recall in `learner_notes.md` alongside your design notes.

Common pitfalls

- Reversing the `[[:SUBCLASS_OF]]` direction.** The hierarchy is asserted as `([:Sichuan]-[:SUBCLASS_OF]->[:Chinese]-[:SUBCLASS_OF]->[:Asian])`. To check “is Sichuan an Asian cuisine?”, the traversal goes `subject -> ... -> object` (outgoing from the subject). Reversing arrows (`<-`) returns no paths, and Stage 2 silently drops to Stage 3 or 4.
- Not parameterizing the Cypher.** F-string interpolation of `subject_id`, `predicate`, or `object_id` into a Cypher string is both a security defect (Cypher injection) and an autograder failure. Use `session.run(query, subject_id=..., object_id=...)`. The only string that may be template-substituted is the relationship-type token after validation against `SCHEMA_CONSTRAINTS`.
- Stage ordering errors.** Entailment must run before contradiction. A valid hierarchical claim like `(recipe:001, OF_CUISINE, cuisine:asian)` where the graph stores the Sichuan link directly would be flagged contradicted if you checked schema constraints before walking the hierarchy. Stage ordering is part of the contract.
- Abstain rate calibration.** If your critic abstains on too many entailed-only claims, recall fails. If it commits too aggressively on claims it cannot justify, precision fails. The thresholds are tuned so a correct cascade clears both — but a swapped Stage 2/Stage 3 ordering or a missed reflexive case will tilt the calibration enough to fail one gate.
- Stage 1 depth-0 vs Stage 2 depth≥1.** For `type` claims, Stage 1 must cover the reflexive case `(subject == object, or [[:SUBCLASS_OF*0..0]])`. Stage 2 must look for depth ≥ 1. If Stage 2 uses `*0..` (the zero-or-more pattern), every supported `type` claim also matches Stage 2 — but Stage 1 fires first, so the entailed-only label remains distinguishable. The cleaner pattern is to use `*1..` in Stage 2 explicitly.

Submit

- Push your branch: `git push -u origin stretch-9b-tue-kg-critic`.
- Open a Pull Request **within your fork** — base `main`, compare `stretch-9b-tue-kg-critic`, **base repository = your fork** (GitHub defaults to the upstream template — change the base-repository dropdown).
- Confirm the autograder workflow runs green on your PR.
- Paste your PR URL into TalentLMS → Module 9 Week B → Stretch — KG Critic to submit this assignment.

